

Case/Topic:-

15/02/24

Computer graphics = Data structures + Graphics algorithm + languages.

Picture generation & basic transformation.

Can

Computer graphics involves pictures or graphics objects which are collection of discrete picture elements called pixels. A pixel is the smallest addressable ~~single~~ ^{screen} element. We can control the pixels by setting the intensity and colour of the pixel that composes the screen.

pixel is not like dot rather pixel is a region that can contain an infinite no. of points.

1) Rasterization: The process of determining the appropriate pixels for representing a picture of graphic subject is called as rasterization.

2) Frame Buffer: eg → ^{Red green} Rgb → blue
Government of West Bengal
It is a large contiguous piece of memory into which the intensity values for all pixels are placed.

3) Bit Map and Pixel Map

If a pixel has only two colour values (black & white), it can be imported by one bit of information. If more than two bits are used to represent the pixel, a larger range of colours or shades of gray can be represented either by 2 bits or 4 bits. An image of two colours is called as a bit map.

An image more than two colours is called as a pixel map.

4) Resolution:

It is defined as the ratio of width and height of output of the display device. There are two types resolution.

1) Image Resolution:

It is defined as the distance from one

School Education Department
pixel to the next pixel.

2) Screen Resolution:

It is defined as the no. of pixels in the horizontal and vertical directions on the screen.

4/05/24

Basic Graphics System

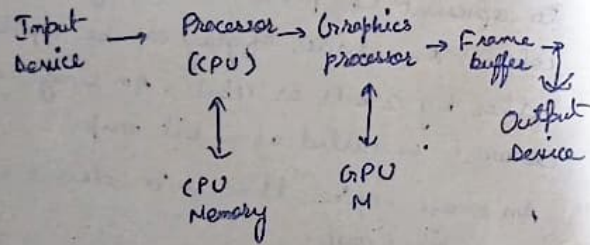
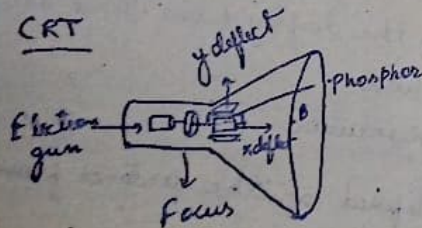


Image formed in frame buffer



Working Principle

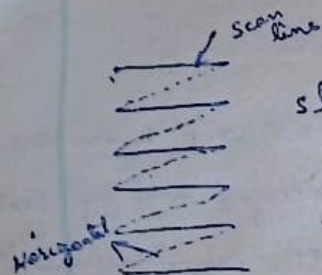
A beam of electrons or e^- CR by an electron gun, passes through focusing and deflection systems that direct the beam towards specified positions on the phosphor coated screen. The phosphor then emits a small spot of light at each position contacted by the ~~electron~~ electron beam.

Scan conversion

The process of representing continuous picture or graphics object as a collection of discrete pixels is called scan conversion.

In CRT monitors there are 2 techniques of displaying images

- 1) Raster scan displays
- 2) Random scan displays



store in the
frame buffer

(RSD) Raster Scan display is based on the television technology.

In the RSD, the electron beam ^{system} swept across the screen, one row at a time from top to bottom as the electron beam moves across each row, the beam intensity is turned on and off to create a pattern of illuminated spots.

RSD

Random / Vector Scan display Unit (RSDU)

When operated as a RSDU, a CRT has the electron beam directed only to the parts of the screen where

a picture is to be drawn.

RSD monitors draw a picture one line at a time and for this reason they are also referred to as vector display or calligraphic display.

The component lines of a picture can be drawn and refreshed by a random scan system in any specified order.

refreshed rate on a random scan system depends on the no. of lines to be display.

Raster Scan display

4. (Picture def. is now stored as the set of line drawing commands in ~~an~~ an area of memory referred to as the refreshed display file or refreshed buffers.)

Operator Interaction

Trackball and spaceball

A track ball is a ball that can be rotated

with fingers or palm of the hand to produce screen cursor movement. potentiometers attached to the ball measure the amount and direction of rotation. Trackballs are often mounted on keyboards or other devices such as Z or mouse.

While a trackball is a 2D positioning device, a space ball provides 6° of freedom. Unlike the track ball, space ball does not actually move.

Strain gauge measure the amount of pressure applied to the space ball to provide input. Spaceballs are used for 3D positioning and selection operations in virtual reality systems, modelling, animation, CAD and other applications.

Joysticks

A joystick consist of a small,

vertical lever mounted on a base that is used to store the screen cursor around. Most joystick select screen position with actual stick movement, others respond to pressure on the stick.

Potentiometers mounted at the base of the joystick measure the amount of movement, and spring returns the stick to the central position when it is released.

Data Glove

Data Glove is used to grasp a virtual object. The glove is constructed with a series of sensors that detect hand and finger motions. Electromagnetic coupling between transmitting antennas and receiving antennas is used to provide information about the position and orientation of the hand.

A 2D projection of the scene can be viewed on a video monitor and or a 3D projection can be viewed with a handset.

- Digitizers
- Light pen.

6/03/24

The Line Drawing Algorithms

Line is ⁱⁿ infinite in dimension.

Line segments have two confined end points.

Slope line equation:

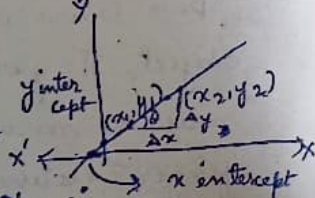
$$y = mx + b \quad \text{--- (2)}$$

m = slope of a line

b = the y intercept of a line

$$m = (y_2 - y_1) / (x_2 - x_1) \quad \text{--- (1)}$$

i.e. $m = \Delta y / \Delta x$



$$m = \frac{\text{perpendicular}}{\text{base}}$$

$$m = \tan \theta$$

Q The end points of line are $(0, 0)$ & $(6, 18)$.
Compute each value of y as x steps from 0 to 6
and plot the result.

Solution $m = y = mx + b$

$$\begin{aligned} m &= (y_2 - y_1) / (x_2 - x_1) \\ &= 18 - 0 / 6 - 0 \\ &= 3 \end{aligned}$$

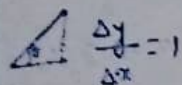
Next the y intercept b is found by plugging y_1 & x_1 into the equation $y = 3x + b$,

$$0 = 3(0) + b.$$

$$b = 0$$

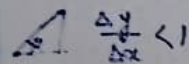
So the ~~line~~ equation for the line
is $y = 3x$

45°



$$\frac{\Delta y}{\Delta x} = 1$$

45°



$$\frac{\Delta y}{\Delta x} < 1$$

45° <



$$\frac{\Delta y}{\Delta x} > 1$$

So which have angle $> 45^\circ$ their slope, will be greater than 1.

DDA Algorithm (Digital Differential Analyzer)

It is an incremental scan conversion method based on calculating either Δx or Δy .

$$\Delta x = \frac{\Delta y}{m} \quad \text{--- (3)}$$

$$\Delta y = m \Delta x \quad \text{--- (4)}$$

We sample the line at unit intervals in one coordinate and determine corresponding integer values nearest the line path for the other coordinate.

Algorithm

(x_1, y_1) (x_2, y_2) are the end points $\Delta x, \Delta y$ are the float variables

where $\Delta x = \text{absolute value } (x_2 - x_1)$ and

$\Delta y = \text{absolute value } (y_2 - y_1)$

(i) if $\Delta x \geq \Delta y$ then

length = Δx []

else

length = Δy

endif

(ii) $x_{\text{increment}} = \Delta x / \text{length}$

$y_{\text{increment}} = \Delta y / \text{length}$

Set pixel (Round(x), Round(y))

iii) for ($K=0$; $K < \text{length}$; $K++$) {

$x++ = x \text{ increment}$

$y++ = y \text{ increment}$

SetPixel (Round(x), Round(y))

}

Considers a first line with positive slope.

If the slope is less than or equal to 1 used sample at unit x intervals [$\Delta x = 1$] and

compute each successive y value as $y_{k+1} = y_k + m$ where x value is 1.

①

For lines with the +ve slope > 1 , we reverse the roles of x and y , i.e. we sample at unit y intervals ($\Delta y = 1$) and calculate each succeeding x value as

$$x_{k+1} = x_k + \frac{1}{m}$$

These two eq^{ns} are based on the

assumptions that lines are to be processed left end to the right end. If the processing is reverse then $\Delta x = -1$ and $\Delta y = -1$

Q Scan convert a line having endpoints (3, 2) & (4, 7) using DDA.

Sol: $dx = x_2 - x_1 = 4 - 3 = 1$

$$dy = y_2 - y_1 = 7 - 2 = 5$$

As, $dx < dy$ then

$$\text{length} = y_2 - y_1 = 5$$

$$dx = (x_2 - x_1) / \text{length} \\ = 1/5 = 0.2$$

$$dy = (y_2 - y_1) / \text{length} \\ = 5/5 = 1$$

$$m = \frac{dy}{dx} \\ = \frac{5}{1} > 1$$

x_1	y_1	x_2	y_2	L	dx	dy	i	x	y
3	2	4	7	5	0.2	1	0		
								1	
								2	
								3	
								4	

~~2.4~~

S End points = $(3, 2)$ & $(4, 7)$

$$\Delta x = 1$$

$$\Delta y = 5$$

$$m = 5 > 1$$

$$\text{length} = dy = 5$$

$$x \text{ increment} = \frac{1}{5} = 0.2$$

$$y \text{ increment} = \frac{5}{5} = 1$$

x_1	y_1	$x \text{ increment}$	$y \text{ increment}$	Result
3	2	$3 + 0.2$ $= 3.2$	$2 + 1$ $= 3$	— $(3, 3)$
		3.4	4	— $(3, 4)$
		3.6	5	— $(3, 5)$
		3.8	6	— $(3, 6)$
		4	7	— $(4, 7)$

↘
This is
our end
point.

10

$$(x_1, y_1) = (2, 5)$$

$$(x_2, y_2) = (2, 102)$$

$$\Delta x = 0$$

$$\Delta y = 7$$

$$m = 7 > 0$$

$$\text{length} = dy = 7$$

$$x \text{ increment} = \frac{0}{7} = 0$$

$$y \text{ increment} = \frac{7}{7} = 1$$

x	y
2	5 — initial
	6

$$(x_1, y_1) = (5, 4)$$

$$(x_2, y_2) = (12, 7)$$

$$\Delta x = 7$$

$$\Delta y = 3$$

$$m = \frac{3}{7} = 0.4$$

$$\text{length} = dx = 7$$

$$\text{increment} = \frac{7}{7} = 1$$

$$\text{fraction} = \frac{3}{2} = 0.42$$

$$\text{steps} = 7$$

x_1	y_1	increment	fraction	Result
5	4	3+1		
		= 4		
		5		
		6		
		7		
10	9	8		

~~Bresenham~~

Bresenham's line drawing Algorithm

An accurate and efficient raster line generating algorithm is developed by Bresenham which converts line using only incremental integer calculations.

$$y = mx + c \quad [x_{\text{next}} = x_{k+1}]$$

$$\text{then, } y = m(x_{k+1}) + c$$

$$d_1 = \text{actual } y - y_k$$

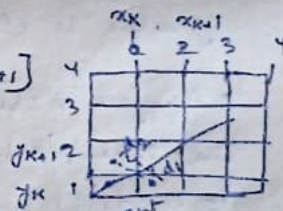
$$= y - y_k$$

$$= m(x_{k+1}) + c - y_k$$

$$= m(x_{k+1}) + c - y_k$$

$$d_2 = y_{k+1} - y$$

$$= y_{k+1} - m(x_{k+1}) - c$$



Find which point is more closer to the line.

• We consider the scan conversion process from lines with +ve slope < 1

• Starting from the left endpoint (x_0, y_0) of a given line, ^{with} step to each successive column (x position) and plot the pixel whose scan line y value is closest to the line path.

• ~~As you~~ Assuming we have determined that the pixel at (x_k, y_k) is to be displayed, we next need to decide which pixel to plot in column x_{k+1} .

• How the choice are positions (x_{k+1}, y_k) and (x_{k+1}, y_{k+1}) ;

• The y coordinate on the line at pixel position x_{k+1} is calculated as

$$y = mx + c$$

if $x = x_{k+1}$

$$y = m x_{k+1} + c$$

~~At~~

At sampling, position x_{k+1} position, we level vertical pixel separations from the original line path as d_1 and d_2 , then

$$\begin{aligned} d_1 &= y - y_k \\ &= (m(x_{k+1}) + c) - y_k \end{aligned}$$

$$\begin{aligned} d_2 &= y_{k+1} - y \\ &= y_{k+1} - m(x_{k+1}) - c \end{aligned}$$

Therefore the difference between two separation is $d_1 - d_2 = [m(x_{k+1}) + c - y_k] - [y_{k+1} - m(x_{k+1}) - c]$

$$\begin{aligned} &= m(x_{k+1}) + c - y_k - y_{k+1} + m(x_{k+1}) + c \\ &= 2m(x_{k+1}) + 2c - y_k - y_{k+1} \end{aligned}$$

$$\Delta x (d_1 - d_2) = \Delta x \left[\frac{2\Delta y}{\Delta x} (x_{k+1}) - 2 \right]$$

=

Decision parameter P_k

$$P_k = \Delta x (d_1 - d_2)$$

$$= \frac{2\Delta y}{\Delta x} x_k - 2\Delta x y_{k+1}$$

of the next decision parameter

$$P_{k+1} = \Delta x (d_1 - d_2)$$

$$= \frac{2\Delta y}{\Delta x} x_{k+1} - 2\Delta x y_{k+1} + C$$

$$P_{k+1} - P_k$$

$$= \frac{2\Delta y}{\Delta x} x_{k+1} - 2\Delta x y_{k+1} + C - 2\Delta y x_k + 2\Delta x y_{k+1}$$

$$= \frac{2\Delta y}{\Delta x} (x_{k+1} - x_k) + C$$

$$P_{k+1} = P_k$$

$$P_{k+1} = P_k + 2\Delta y (x_{k+1} - x_k) - 2\Delta x y_{k+1}$$

where the term $y_{k+1} - y_k$ is either 0 or 1 depending on the sign of parameter P_k

Bresenham Algorithm

- 1) Input the two line endpoints and store the left end point in (x_0, y_0) .
- 2) Load (x_0, y_0) into the frame buffer i.e. plot the first point.
- 3) Calculate constants Δx , Δy , $2\Delta y$, $2\Delta y - 2\Delta x$ and obtained the starting value for the decision parameter as $p_0 = 2\Delta y - \Delta x$.
- 4) At each x_k along the line, starting

at $k=0$ perform the following test

a) if $P_K \leq 0$

the next point to plot is x_{K+1}, y_K

$$x_{K+1} = P_K + 2\Delta y$$

b) else

the next point to plot is

$$x_{K+1}, y_{K+1}$$

and

$$P_{K+1} = P_K + 2\Delta y - 2\Delta x$$

5) Repeat step 4 Δx times.

Example

Two line endpoints are

$(20, 10)$ and $(30, 18)$

this has the slope of 0.8 with $\Delta x = 10$ and $\Delta y = 8$.

Calculate the successive line positions

$$\begin{aligned} P_0 &= 2\Delta y - \Delta x \\ &= 16 - 10 \\ &= 6 \end{aligned}$$

$$\begin{aligned} P_1 &= 6 + 2 \cdot 16 \\ &= 22 \end{aligned}$$

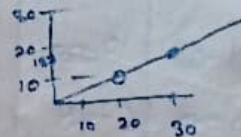
$$\begin{aligned} P_1 &= 6 + 16 - 16 \\ &= 6 \\ P_2 &= \end{aligned}$$

$$\begin{aligned} P_1 &= 6 + 16 - 20 \\ &= 2 \end{aligned}$$

$$x_{K+1} = 20 + 1 = 21$$

$$y_{K+1} = 10 + 1 = 11$$

P_K	P_{K+1}	x_{K+1}	y_{K+1}
6	22	20	10
6	2	21	11
2	-2	22	12



$$\begin{aligned} P_2 &= 2 + 16 - 20 \\ &= -2 \\ P_3 &= -2 + 16 - 20 \\ &= -6 \end{aligned}$$

A	P_{K+1}	x_{K+1}	y_{K+1}
-2	14	23	12
14	10	24	13
10	6	25	14
6	2	26	15
2	-2	27	16
-2	14	28	16
10	10	29	17
14	6	30	18

$$-2 + 16 - 24$$

$$= -2 - 4$$

$$= -6$$

$$-6 + 16 - 20$$

$$= -6 - 4$$

$$= -10$$

P_K	P_{K+1}	x_{K+1}	y_{K+1}
10	6	25	14
6	2	26	15
2	-2	27	16
-2	14	28	16
14	10	29	17
10	6	30	18

14/03/24

Mid point Circular (Algorithm)

• Input radius are an circle center (x_c, y_c) and obtained the 1st point on the circumference of a circle centered on the origin as $(x_0, y_0) = (0, r)$



• calculate the initial value of the decision parameter as

$$p_0 = \frac{5}{4} - r$$

• At each x_K position, starting at $K=0$

P_K	P_{K+1}	x_{K+1}	y_{K+1}
		20	10
6	2	21	11
2	-2	22	12
-2	14	23	12
14	10	24	13

perform the following test:

i) If $p_k < 0$, then the next point along with the circle centered on $(0,0)$ is (x_{k+1}, y_k)

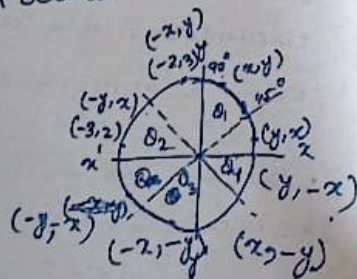
$$\text{and next } p_{k+1} = 2p_k + 2x_{k+1} + 1$$

ii) ~~also~~ the next point along the circle is

$$(x_{k+1}, y_{k+1}) \text{ and } p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

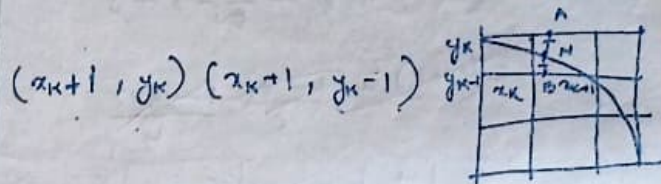
$$\text{where } 2x_{k+1} = 2x_k + 2 \text{ and } 2y_{k+1} = 2y_k - 2$$

• Determine symmetry points in the other ~~eight~~ seven ~~other~~ octants.



• Move each calculated pixel position (x, y) on to the circular path centered on (x_c, y_c) and plot the coordinate values, $x = x + x_c$; $y = y + y_c$

• Repeat steps 3 to 5 until $x \geq y$.



$$\text{Mean Mid} = \left(\frac{x_k + 1 + x_k + 1}{2}, \frac{y_k + y_k - 1}{2} \right)$$

$$= (x_{k+1}, y_k - \frac{1}{2})$$

$$\text{Circle formula} = x^2 + y^2 = r^2$$

$$\text{or } x^2 + y^2 - r^2 = 0$$

circle eqⁿ at N point

$$\text{Circle f/circle} = \frac{(x_{k+1}, y_k - \frac{1}{2})}{p_k} = (x_{k+1})^2 + (y_k - \frac{1}{2})^2 - r^2$$

$$P_k = (x_k + 1)^2 + y_k^2 + \frac{1}{4} - y_k - x^2$$

~~Ref: (0, 1)~~

$$P_k = (0, 1)$$

$$= (0+1)^2 + \cancel{1^2} + \frac{1}{4} - \cancel{1} - \cancel{0^2}$$

$$= \cancel{(0+1)^2} + \frac{\cancel{1-1^2}}{4} + \frac{\cancel{0-0}}{4}$$

$$= 1 + \frac{1}{4} - 1 = \boxed{\frac{5}{4} - 1}$$

2/10/24

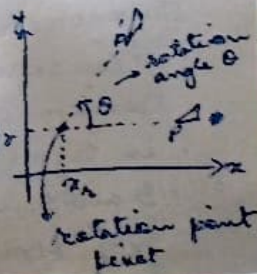
Object transforms with the scaling eqn
 $x' = p * x$ are used for both scaled and repositioned. Scaling factors with ^{values} less than 1 move objects closer to the coordinate origin while greater than 1 move coordinate positions further from the origin.

- When s_x and s_y assigned the same values are uniform scaling is produced that maintains relative object proportions.

2D-Rotation

1. ~~On~~ 2D rotation is applied to an object by repositioning it along a circular path in the (x, y) plane.

- 2/ Rotation angle is θ and the position of rotation point (x_0, y_0) (pivot point) above which the object is to be



rotated.

3 + values of rotation angle means counter clockwise rotation.

4 - values of rotation angle means clockwise rotation.

5 First we determine the transformation eqⁿs for rotation of a point P.

a) Here pivot point is at the coordinate origin.

$$\sin \theta = \frac{\text{perpendicular}}{\text{hypotenuse}}$$

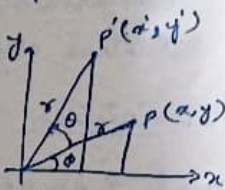


Fig Rotation of a point from (x, y) to (x', y') through an angle θ relative to coordinate origin.

The original angular displacement

b) Using standard trigonometric identities we can express the transform coordinates in terms of angle θ and ϕ as—

$$x' = r \cos \phi$$

of the point from the x-axis is ϕ .

$$y = r \sin \phi \quad \text{--- (2)}$$

$$x' = r \cos (\phi + \theta) \quad \text{--- (1)}$$

$$x' = r \cos \phi \cos \theta - r \sin \phi \sin \theta$$

$$y' = r \sin (\phi + \theta) \quad \text{--- (2)}$$

$$= r \cos \phi \sin \theta + r \sin \phi \cos \theta$$

c) Substituting eqⁿ(2) into eqⁿ(1), we get

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

d) Rotation matrix will be $P' = R \cdot P$ where

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

$$\text{eg } P(4, 3)$$

$$\text{angle} = 45^\circ$$

$$\theta = 45^\circ$$

$$x = \frac{1}{\sqrt{2}}$$

$$x' = x \frac{1}{\sqrt{2}} - y \frac{1}{\sqrt{2}}$$

$$= \frac{4-3}{\sqrt{2}} = \frac{1}{\sqrt{2}}$$

$$y' = \frac{1}{\sqrt{2}} (x+y)$$

$$= \frac{7}{\sqrt{2}}$$

$$P' \left(\frac{1}{\sqrt{2}}, \frac{7}{\sqrt{2}} \right)$$

Unit-4

Width: Implementation of line width options depends on the capabilities of the output device.

Line width parameter lw is assigned a positive number to indicate the relative width of the line to be displayed. A value of 1 specifies a standard width line. For example on a pen plotter, for instance, a user could set lw to a value of 0.5 to plot a line whose width is half of the standard line.

- value > 1 produce lines thicker than the standard.

- For raster implementation, a

standard width line is generated with single pixels at each sample position.

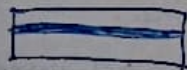
Line Endcaps

The problem with implementing with options using horizontal or vertical pixel spans is that the method produces line whose ends are horizontal or vertical regardless of the slope of the lines.

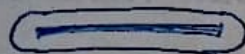
This effect is more noticeable

We can ~~also~~ adjust the shape of line ends to give them a better appearance by ~~adjusting~~ adding line caps.

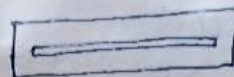
Butt cap



Round cap



Projecting square cap



The butt cap obtained by adjusting the end positions of the component parallel lines so that the thick line is displayed with ~~so~~ square ends that are perpendicular to the line path.

Another round cap is obtained by adding a filled semicircle to each butt cap. The circular arc are centered on the line endpoints and have a diameter = line thickness.

The third type of line cap is the projecting square cap. Here we simply extend the line and add butt caps that positioned 1/2 of the line width beyond the specify endpoints.

Line Joins

Miter Join



Round Join



Bevel Join



There are three possible methods for smoothly joining two line segments. These are:

A miter Join is accomplished by extending the outer boundaries of each of the two lines until they meet.

A Round Join is produced by capping the connection between the two segments with a circular boundary whose diameter is = line width.

A Bevel Join is generated by displaying

the line segments with both caps and filling in the triangular gap where the segment meets. If the angle between the two connected line segments is very small, a miter join can generate a long spikes that distort the appearance of the polygons. To avoid this we can switch from miter join to round or Bevel Join.

Pen and Brush

- The selected "pen" or "brush" determine the way a line will be drawn.
- Pens and brushes have size, shape, color and pattern attribute.
- pixel mask is applied in both of them.

Curve Attribute

We can display curves with varying color, widths, dot hyphen dash and available pen and brush options.

Later curves of various width can be displayed using the method of horizontal or vertical pixel spans. If the magnitude of the curve slope < 1 , we plot vertical spans. If the magnitude of the slope > 1 we plot horizontal spans.

* Two ways to store information:

1. Direct
2. Indirect.

22/04/24

Homogeneous ~~Native~~ Coordinate of Matrix Representation

$P' = M_1 P + M_2$ → general equations
of

$P' = P + T$ → Translation

$P' = P \cdot S$ or $P \cdot R$ → Rotation

- P and P' are column vectors
- M_1 is a 2 by 2 array containing multiplicative factors.

• M_2 is a 2 element column matrix containing translational terms

• For translation M_1 is identity Matrix

• For rotation or scaling, M_2 contains the translational terms associated with the pivot point or scaling fixed point.

• Multiplicative and translational terms for a 2D g

⋮
⋮
⋮
⋮
⋮
⋮

• Expand each 2D coordinate (x, y) to three element representation (x_h, y_h, h) called homogeneous coordinates

• h is the homogeneous parameter such that $x = x_h/h$ $y = y_h/h$.

• → infinite homogeneous representations

for a point

i. A convenient choice is to choose $h=1$.

• 2D Translation Matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\Rightarrow P' = T(t_x, t_y) \cdot P$$

• 2D Rotation Matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

This the matrix for

$$\text{or, } P' = R(\theta) \cdot P$$

• 2D Scaling Matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\text{or, } P' = S(s_x, s_y) \cdot P$$

2D Composite Transformations

• We can select a sequence of transformations as a composite transformation matrix by calculating matrix product of the individual transformations

$$P' = M_2 \cdot M_1 \cdot P \\ = M \cdot P$$

Composite 2D Translations

• If two successive translation are applied to a point P , then the final transformed location P' is calculated as

$$P' = T(t_{x2}, t_{y2}) \cdot T(t_{x1}, t_{y1}) \cdot P = T(t_{x1} + t_{x2}, t_{y1} + t_{y2}) \cdot P$$

$$\begin{bmatrix} 1 & 0 & t_{x2} \\ 0 & 1 & t_{y2} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & t_{x1} \\ 0 & 1 & t_{y1} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_{x1} + t_{x2} \\ 0 & 1 & t_{y1} + t_{y2} \\ 0 & 0 & 1 \end{bmatrix}$$

Composite 2D Rotations

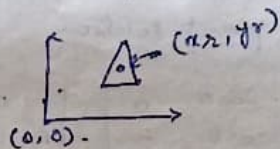
$$P' = R(\theta_1 + \theta_2) \cdot P$$

$$\begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 \\ \sin \theta_2 & \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

General pivot point RotationSteps

1. Translate the object so that the pivot point is moved to the coordinate origin.
2. Rotate the object about origin.
3. ^{Translate} ~~Rotate~~ the object so that the pivot point is returned to its original position.

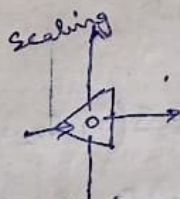


(a)

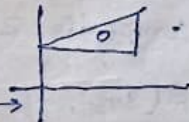
Original position of object and pivot point.



Translation of object so that pivot point (x_r, y_r) is at origin



Translation of object so that the pivot point return to the position (x_r, y_r)



General 2D point-point rotation

$$\begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta & -\sin \theta & x_r(1 - \cos \theta) + y_r \sin \theta \\ \sin \theta & \cos \theta & y_r(1 - \cos \theta) - x_r \sin \theta \\ 0 & 0 & 1 \end{bmatrix}$$

Composite 2D scaling

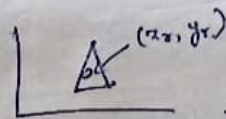
$$S(sx_2, sy_2) \cdot S(sx_1, sy_1) = S(sx_1 \cdot sx_2, sy_1 \cdot sy_2)$$

$$\begin{bmatrix} sx_2 & 0 & 0 \\ 0 & sy_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} sx_1 & 0 & 0 \\ 0 & sy_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} sx_1 \cdot sx_2 & 0 & 0 \\ 0 & sy_1 \cdot sy_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

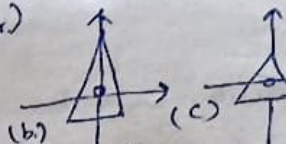
General fixed point scaling

Steps

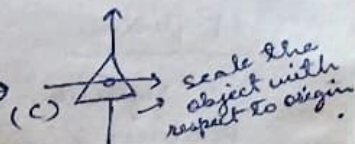
1. Translate the object so that the fixed point coincides



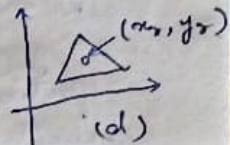
(a)
Original position
of object and
fixed point



(b)
Translate
object
so that
fixed point
(x_r, y_r)
is at origin



scale the
object with
respect to origin



Translate object
so that the
fixed point
(x_r, y_r) is
return to
original
position.

General 2D fixed-point scaling:

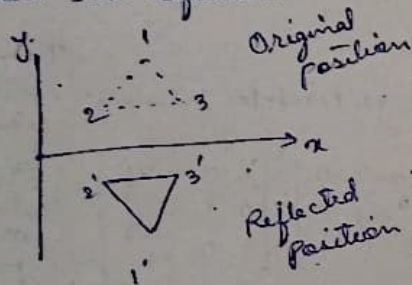
$$\begin{bmatrix} 1 & 0 & x_f \\ 0 & 1 & y_f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} sx & 0 & x_f(1 - sx) \\ 0 & sy & y_f(1 - sy) \\ 0 & 0 & 1 \end{bmatrix}$$

$$T(x_f, y_f) \cdot S(sx, sy) \cdot T(-x_f, -y_f) = S(sx, sy, sx, sy)$$

Reflection

A reflection is a transformation that produces a mirror image of an object. The mirror image for a 2D reflection is generated relative to an axis of reflection by rotating the object 180° about the reflection axis.



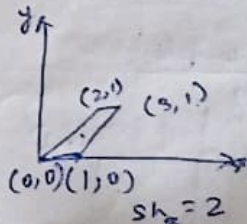
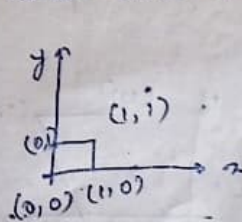
H.W Reflect the diagram about y-axis and write the matrix reflection about y-axis.

• Reflection about the line $y=0$ (the x-axis)

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shear

• A transformation that distorts the shape of an object such that the transformed shape appears as the object, was composed of internal layers that had been caused to slide over each other is called a shear.



Two common shearing transformations are those that shift coordinate x -value and that shift y values.

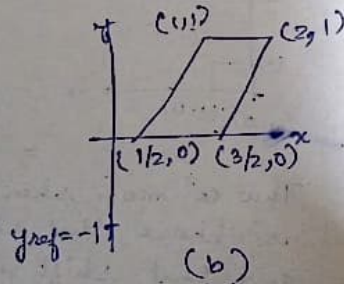
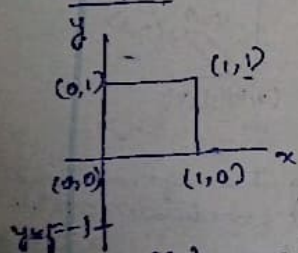
Shear

- An x -direction shear relative to the x -axis

$$\begin{bmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{aligned} x' &= x + shx \cdot y \\ y' &= y \end{aligned}$$

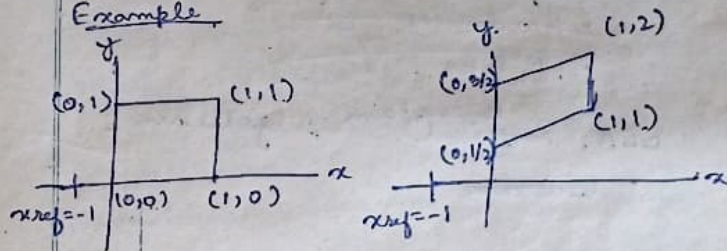
- An y -direction shear relative to the y -axis

$$\begin{bmatrix} 1 & 0 & 0 \\ shy & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Example

A unit square (a) is transformed to a shifted parallelogram

(b) with $shx = 0.5$ and y

Example

A unit square (a) is turned into a shifted parallelogram

(b) with parameter values $shy = 0.5$ and $x_{ref} = -1$ in the y -direction shearing transform

Topic1st chapter

- graphics
- definitions
- I/O devices
- CR Tube

DDA, ~~Raster~~ Bresenham Algorithm

~~Circle~~

Chapter 62D ViewsWindow

A world coordinate area selected for display is called a window.

Viewport

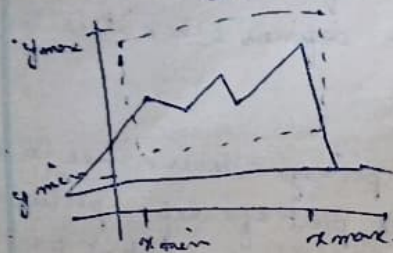
An area on a display device to which

a window is mapped is called a viewport.
The window defines what is to be viewed,
the viewport defines where it is to be displayed.

A graphics package allows a user to specify which part of a defined picture is to be displayed and where that part is to be placed on the display device. Any conventional cartesian coordinate system, referred to as the world coordinate reference frame, can be used to define the picture. For a 2D picture, a view is selected by specifying a subarea of the total picture area.

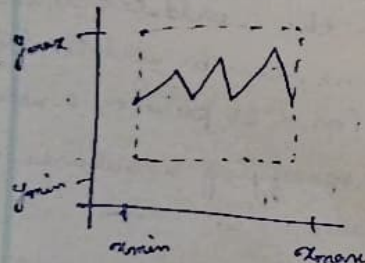
The mapping of a part of a world coordinate scene to device coordinates is referred to as a viewing transformation. Sometimes this transformation is called window-~~to~~ viewport transformation or the windowing.

transformation window



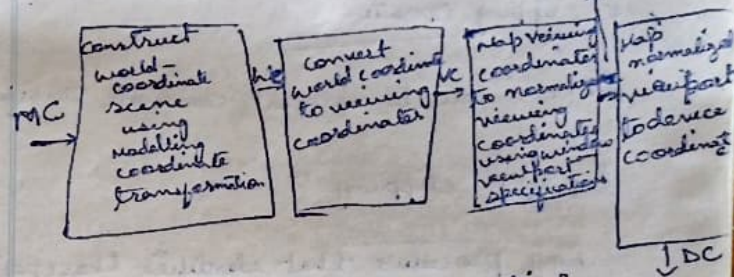
world coordinates

viewport



device coordinates

The viewing transformation is done in the following steps:



The 2D viewing transformation Pipeline

29/04/24 * Study Output devices

20/04/21

Clipping Operation

Q. Window to viewport coordinate transformation.

Q. What is clipping?

Any procedure that identifies those portion of a picture that are either inside or outside of a specified region of space is referred to as a clipping algorithm or simply clipping.

The region ~~come~~ against which an object is to be clipped is called a clip window.

Clipping algorithms can be applied in world coordinates so that only contents of the window interior are mapped to device coordinate.

Here we consider the following

clipping algo. of the following parameters
types

- 1) Point clipping
- 2) line ^{clipping} (straight line segment)
- 3) Area clipping (Polygons)

Point clipping

~~Ass~~ Assuming that the clip window is a rectangle in standard position, we say a point $p = (x, y)$ for display if the following inequalities are satisfied.

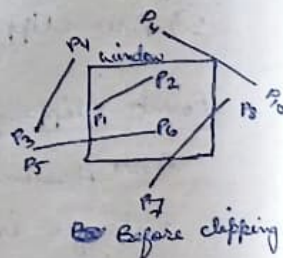
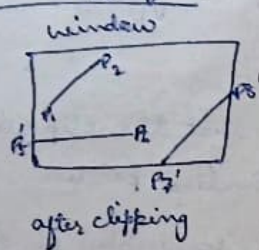
$$x_{wmin} \leq x \leq x_{wmax}$$

$$y_{wmin} \leq y \leq y_{wmax}$$

where the edges of clip window x_{wmin} , y_{wmin} , x_{wmax} , y_{wmax} can be either the world coordinate window boundaries or viewport boundaries.

If anyone of these 4 equalities is not satisfied, the point is clipped (not same for display).

Line Clipping



A line clipping procedure involves several parts. First we can ~~test~~ ^{given} test the line segment to determine whether it lies completely inside the ^{clipping} ~~given~~ window if it does not, we try to determine whether it lies completely outside the window.

2) If we cannot identify a line as

completely outside or completely inside, we must perform intersection calculations with one or more clipping boundaries. We process lines through the inside-outside test by checking the line end points.

4) A line with ~~points~~ ^{inside} both endpoints ~~outside~~ ^{outside} all clipping boundaries such as the line P_1 to P_2

5) A line with both endpoints outside anyone of clipping boundaries such as the P_3 to P_4 is outside the window.

6) for a line segment with endpoints with (x_1, y_1) & (x_2, y_2) one or both endpoints outside the clipping window, the parametric representation $x = x_1 + U(x_2 - x_1)$

$$y = y_1 + U(y_2 - y_1)$$

$$0 \leq U \leq 1$$

The parametric eqⁿ could be use to determine the value of parameter U for intersection with the clipping boundary coordinate. If the value of U for an intersection with a k rectangle boundary is outside the range 0 to 1 the line does not enter the interior of the window. If the value of U is within the range from 0 to 1, the line segment does cross into the clipping area. This method can be applied to each clipping boundary ^{in order to determine} whether ~~if~~ any part of line is to be displayed.

21/05/21

Structure and Hierarchical

A ~~set~~ ^{label} set of output primitives ⁱⁿ PHIGS is called a structure. Other commonly used labelled collection of primitives are segments and objects.

Basic Structure functions

- Open structure & Close
- Close structure

Any no. of structures can be created for a picture but only one structure can be open at a time. any ~~an~~ open str. must be closed before a new structure can be created.

Interactive Input MethodsLogical Classification of input devices

The various kind of input data are summarized into following 6 logical used by PHIGS and GKS.

1. LOCATOR: A device for specifying a coordinate position (x, y).
2. STROKE: A device for specifying a series coordinate positions.
3. STRING: A device for specifying text input.
4. VALUATOR: A device for specifying scalar values.
5. CHOICE: A device for selecting many options.

6. PICK: A device for ^{selecting} picture content component.

Input Functions

Graphical input functions can be setup to allow users to specify the following options:

1. Which physical devices are to provide input within a particular logical classification for example a tablet used as stroke device.
2. How the graphics program and devices are to interact. For example either the program or the devices can initiate data entry or both can operate simultaneously.
3. When the data are to be input and which devices is to be used at that time to deliver a particular input type to the specified data variables.

Input modes

The input modes specify how the program and input devices interact.

Input could be initiated by the program, or the program and input devices both could be operated simultaneously or data input could be initiated by the devices.

These three input modes are referred to as request mode, sample mode and event mode.

In request mode, the application program initiates data entry. Input values are requested and processing is suspended until the required values are received.

In sample mode, the application program and input devices operate independently.

In event mode, the input devices initiate data ^{input} into the application program.

The program and the input devices again operate concurrently, but now the input devices deliver data to the input queue.

Intuitive Colour Model ConceptsRGB Colour ModelYIQ Colour ModelCMY Colour Modelgray colour model